Software Requirements Specification

YouSightSeeing App

1. Introduction

YouSightSeeing App is a mobile application for automatic tourist route generation based on user preferences and current route parameters. The application helps users authorize through Google, manage interests, view a map, generate tourist routes, edit selected points, save routes, and reuse saved routes later.

The current backend version supports route calculation, recommendation-based route generation, persistent saved routes, route point storage, user preference weights, and event-based recommendation updates. Saved routes and user actions influence future recommendations.

2. Glossary

- GPS - Global Positioning System, used to determine the user location.
- Point of Interest (POI) - an object on the map that may be useful for tourists, such as a park, sight, museum, cafe, or landmark.
- Route - a sequence of selected POIs and a pedestrian path calculated for visiting them.
- Route geometry - the line of a calculated route returned by the route calculation service as coordinates in [longitude, latitude] format.
- Saved route - a route stored in the database together with its metadata and ordered POI list.
- Route point - a specific saved POI inside a saved route, including position, place_id, name, address, categories, and coordinates.
- User preference - a category weight from 0 to 1 that represents the user interest in a category.
- User event - an action such as place_viewed, route_generated, route_opened, route_saved, or route_completed, used by the recommendation system.
- Recommendation score - a numerical value used by the backend to compare candidate POIs during route generation.

3. Actors

3.1 User

Role: End user of the mobile application.

Goals: Authorize, set preferences, generate routes, edit selected points, save routes, view saved routes, and receive better recommendations over time.

Responsibilities: Provide correct route parameters, allow location access if needed, select or edit route points, and follow application recommendations at their own discretion.

3.2 Google Authentication Service

Role: External authentication provider.

Responsibilities: Validate user Google token and provide identity data used by the backend to create or load the user profile.

3.3 Geoapify

Role: External POI provider.

Responsibilities: Return candidate POIs near the selected start point according to categories, radius, and limit.

3.4 OpenRouteService

Role: External route calculation provider.

Responsibilities: Build pedestrian routes and route matrix data used for route generation and recalculation.

3.5 Yandex MapKit

Role: Client-side map SDK.

Responsibilities: Display the map, user location, POI markers, and calculated route line.

4. Functional Requirements

4.1 Strategic Use Cases

Use Case UC-S-1: Registration and Onboarding

Actors: User, Google Authentication Service, System.

Goal: Authenticate the user and prepare a profile for personalized recommendations.

Preconditions: The application is installed, internet connection is available, and the user has a Google account.

Trigger Condition: First application launch or explicit login action.

Extensions: UC-S-2 Route Preference Management, UC-S-3 Recommended Route Generation.

Main Success Scenario:

1. User opens the application.
2. Application displays the Google authentication option.
3. User completes authorization through Google.
4. Client sends Google token to the backend.
5. Backend validates the token and creates a new user or loads the existing user.
6. Backend returns access token, refresh token, and user profile data.
7. Application opens the main screen with the map.
8. If preferences are not configured yet, application offers the user to choose preferred categories.

Alternative Scenarios:

- Authentication failed - the system displays an authentication error and allows the user to retry.
- Internet connection lost - the system displays a network error and waits for reconnection.
- Invalid token - the backend returns an authorization error and the client requests a new login attempt.

Use Case UC-S-2: Route Preference Management

Actors: User, System.

Goal: Store and update the user interest profile used by recommendations.

Preconditions: User is authenticated.

Trigger Condition: User opens preference settings or completes onboarding preferences.

Extensions: UC-S-3 Recommended Route Generation, UC-S-6 Recommendation Feedback.

Main Success Scenario:

1. User opens preference settings.
2. Application displays available categories and current weights if they exist.
3. User changes preferred categories or interest levels.
4. Client sends updated preferences to the backend.
5. Backend validates category names and weights.
6. Backend saves the updated category weights.
7. Application uses updated preferences in future route generation.

Alternative Scenarios:

- Invalid weight - the backend rejects values outside the allowed range 0 <= weight <= 1.
- Empty category - the backend rejects a preference item without category.
- Unauthorized request - backend returns an authorization error.

Use Case UC-S-3: Recommended Route Generation

Actors: User, Geoapify, OpenRouteService, Yandex MapKit, System.

Goal: Generate a pedestrian route based on current parameters, user preferences, and recommendation history.

Preconditions: User is authenticated or uses a test/debug request in local development. Internet connection is available for Geoapify and OpenRouteService.

Trigger Condition: User presses the route generation button.

Extensions: UC-S-4 Route Editing and Recalculation, UC-S-5 Route Saving.

Main Success Scenario:

1. User selects or confirms a start point on the map.
2. User configures route parameters: categories, radius, maximum number of places, desired duration in minutes, and whether to include food places.
3. Client sends the route generation request to the backend.
4. Backend validates coordinates, radius, maximum number of places, and duration.
5. Backend loads saved user preference weights.

6. If the user explicitly selected categories, these categories receive the highest priority for the current generation. If categories are empty, backend derives categories from stored preference weights.
7. Backend requests Geoapify for candidate POIs near the start point.
8. Backend filters candidates: invalid coordinates, duplicates, low-quality places, near duplicates, food places when include_food is false, and excessive repetition of one class are removed.
9. Backend calculates candidate scores using preference weight, distance from start, food penalty, and place-level history bonuses or penalties.
10. Backend requests an OpenRouteService matrix to estimate travel time between candidates.
11. Backend greedily selects points using score/time gain and route constraints.
12. Backend applies local 1-swap improvement to replace selected points if a better valid combination exists.
13. Backend calls route calculation to build the final pedestrian route geometry.
14. Backend stores route_generated events for selected places.
15. Backend returns selected places and route data to the client.
16. Client displays markers and route line on the map.

Alternative Scenarios:

- No suitable POIs found - the backend returns an error and the application suggests increasing radius, duration, or category coverage.
- OpenRouteService matrix error - the backend returns route matrix calculation error.
- OpenRouteService route calculation error - the backend returns route calculation error.
- Geoapify error - the backend returns place search error.
- Invalid coordinates or radius - the backend returns validation error.

Use Case UC-S-4: Route Editing and Recalculation

Actors: User, OpenRouteService, Yandex MapKit, System.

Goal: Allow the user to change selected route points and rebuild the route line.

Preconditions: A route was generated or points were selected manually.

Trigger Condition: User opens route editing mode.

Extensions: UC-S-5 Route Saving.

Main Success Scenario:

1. User views the generated route and selected POIs.
2. User modifies the route by removing points, adding available POIs, or changing the point order.
3. Client sends the ordered coordinate list to the backend route calculation endpoint.
4. Backend validates that there are at least two route coordinates.
5. Backend calls OpenRouteService to calculate pedestrian route geometry.
6. Backend returns route geometry, distance, and duration.
7. Client redraws the route line and updated markers on the map.
8. User accepts the final route or continues editing.

Alternative Scenarios:

- Not enough points - backend returns an invalid route points error.
- External routing service unavailable - backend returns route calculation error.
- Selected points create an impractical route - the application offers the user to remove points or change order.

Use Case UC-S-5: Route Saving

Actors: User, System.

Goal: Persist the final route and use it as recommendation feedback.

Preconditions: User is authenticated and a generated or manually edited route exists.

Trigger Condition: User presses the Save Route button.

Extensions: UC-S-6 Saved Route Viewing, UC-S-7 Recommendation Feedback.

Main Success Scenario:

1. Client sends route metadata and ordered route points to the backend.
2. Backend reads the current user id from authorization context.
3. Backend validates that the request contains route points.
4. Backend maps the request into Route and RoutePoint entities.
5. Backend starts a database transaction.
6. Backend saves route metadata to the routes table.
7. Backend saves every route point to the route_points table, preserving position, place_id, categories, coordinates, name, and address.
8. Backend creates route_saved events for saved route points.
9. Backend updates recommendation category weights according to the saved point categories.
10. Transaction is committed.
11. Backend returns the saved route identifier to the client.

Alternative Scenarios:

- User is not authenticated - backend returns an authorization error.
- Route points are missing - backend returns an invalid route points error.
- Database error while saving route or points - transaction is rolled back and backend returns an error.
- Recommendation event creation error - transaction is rolled back to avoid partial saving.

Note: the current saved route model stores route metadata and ordered POI points. The detailed route line can be displayed from the generated route response while the user is on the current screen, or recalculated later from saved start coordinates and saved route points if needed.

Use Case UC-S-6: Saved Route Viewing

Actors: User, System, Yandex MapKit, OpenRouteService if route line recalculation is required.

Goal: Allow the user to view saved routes and restore route details.

Preconditions: User is authenticated and has at least one saved route.

Trigger Condition: User opens the saved routes screen or a specific saved route.

Main Success Scenario:

1. User opens the saved routes screen.
2. Client requests the list of saved routes from the backend.
3. Backend returns only routes belonging to the current user.
4. User selects one saved route.
5. Client requests the selected route by id.
6. Backend checks route ownership or public availability.
7. Backend returns route metadata and ordered route points.
8. Client displays saved POI markers and route details.
9. If a route line is needed, client can request route recalculation using saved start coordinates and saved ordered points.

Alternative Scenarios:

- Route does not belong to the user and is not public - backend returns not found or access error.
- Route id has invalid format - backend returns invalid route id error.
- User has no saved routes - application displays an empty saved routes state.

Use Case UC-S-7: Recommendation Feedback

Actors: User, System.

Goal: Improve future recommendations based on user actions and saved routes.

Preconditions: User is authenticated.

Trigger Condition: Application sends a user event or backend creates an event after route generation or route saving.

Main Success Scenario:

1. A user action occurs: place viewed, route generated, route opened, route saved, or route completed.
2. Backend creates a user_events record with event type, optional route id, optional place_id, and optional category.
3. If the event has a category and the event type changes category preferences, backend recalculates the category weight.
4. New weight is saved to user_category_preferences.
5. During the next route generation, backend loads category weights and recent place events.
6. Saved or viewed points can receive a bonus, while repeatedly generated or completed points can receive a repetition penalty.
7. Generated route becomes more personalized and less repetitive.

Weight update formula:

newWeight = oldWeight + delta * (1 - oldWeight)

Current event deltas:

- place_viewed: +0.03
- route_opened: +0.04
- route_saved: +0.12

- route_completed: +0.08
- route_generated: +0.00

Place-level score adjustments:

- route_generated adds a repetition penalty for the same place_id.
- route_completed adds a smaller penalty because the user has already visited the place.
- place_viewed, route_opened, and route_saved add bonuses.
- Total penalty and bonus are capped to prevent permanent exclusion or permanent domination of a point.

5. API-Level Functional Coverage

The backend exposes the following main API groups:

- Auth: Google login, refresh token, logout.
- Users: get/update current user, update profile picture, get/update user preferences.
- Places: search POIs through Geoapify.
- Routes: calculate route, generate recommended route, save route, get saved route by id, get saved route list.
- Events: track user actions for recommendation feedback.

Debug endpoints with X-Test-User-ID may be used only for local development and testing. Production clients should use authenticated /api endpoints with Bearer JWT.

6. Non-Functional Requirements

6.1 Environment

Server Side:

- Programming language: Go.
- HTTP framework: Echo.
- API style: REST API over HTTP/HTTPS.
- Database: PostgreSQL.
- Persistence: users, refresh tokens, saved routes, route points, user events, and user category preferences.

Client Side:

- Platform: Android.
- Programming language: Java.
- Map SDK: Yandex MapKit.
- Minimum Android version: 8.0, API level 26.

External Services:

- Google API - user authentication.
- Geoapify API - POI search.
- OpenRouteService API - route matrix and pedestrian route geometry.

- Yandex MapKit - map display, location display, markers, and route visualization.

6.2 Performance

- Common profile and preference operations should complete within 1 second under normal conditions.
- Route generation should normally complete within several seconds, depending on Geoapify and OpenRouteService response time.
- Route calculation should return distance, duration, and geometry with minimal additional backend processing.
- Saved route list retrieval should support pagination through limit and offset.
- Backend should limit maximum route places to avoid excessive external API calls.

6.3 Reliability

- Backend must validate input data before calling external providers.
- Route saving must be transactional: route metadata, route points, and recommendation events should not be partially saved.
- Backend must log internal errors without exposing sensitive details to the client.
- If Geoapify or OpenRouteService is unavailable, backend should return a clear API error and client should show a user-friendly message.
- Database backups should be configured for production deployment.

6.4 Security and Privacy

- Private API endpoints must require Bearer JWT authentication.
- Saved routes must be associated with the authenticated user.
- Users must not access private saved routes of other users.
- Refresh tokens must be stored securely using hashes.
- User personal data must be transmitted through HTTPS in production.
- Debug endpoints must not be publicly exposed in production deployments.

6.5 Extensibility

- The recommendation system must allow adding new POI categories and new event types.
- The route provider should be replaceable if another routing service is required.
- Additional POI providers can be integrated besides Geoapify.
- Route sharing can be extended through public routes and share codes.
- Saved route geometry can be added later if route lines must be restored without recalculation.
- API documentation should be maintained through OpenAPI.

7. Current Implementation Notes

- Generated route responses contain selected places and calculated route geometry.
- Saved route responses contain saved route metadata and ordered POI points.
- Saving a route creates route_saved events and updates category weights.
- Repeated generated points receive place_id penalties during future route generation.

- Explicitly selected categories have priority over stored preferences for the current generation request.
- If no categories are provided, the backend derives preferred categories from stored user weights and defaults to tourism.sights and leisure.park if no strong preferences exist.